FORMAL LANGUAGE & AUTOMATA THEORY

Academic Microproject Report (Curriculum Modules 1, 2, and 3)

Student Name: Kunal Yadav

Roll Number: 2411021060008

Squad / Cohort: Squad 70

Course: FLAT (Automata Theory)

Academic Level: Undergraduate Engineering

Date: October 2026

1. Project Overview & Curriculum Alignment

This project introduces a comprehensive, web-based educational visualizer and algorithmic simulator for Formal Language and Automata Theory. Built completely from foundational first principles using standard web technologies (HTML5, CSS3, ES6 JavaScript, SVG) without any third-party framework or runtime dependency, the application bridges theoretical abstraction with real-time interactive computation.

Curriculum Module	Topics Addressed	Implemented Pedagogical Feature
Module 1: Formal Languages & RegEx	Alphabets, strings, operations on languages, transition systems, regular expressions.	Interactive Σ^k string generator, prefix/suffix/reversal tool, language union/concatenation/Kleene star, and Shunting-Yard parser.
Module 2: Finite Automata &	DFA, NFA, equivalence, Thompson's	Interactive step-by-step string tracer with tape head pointer, Thompson ε-NFA

Curriculum Module	Topics Addressed	Implemented Pedagogical Feature
Minimization	construction, subset construction, DFA state minimization.	builder, subset powerset derivation table, and Myhill-Nerode table-filling matrix.
Module 3: CFG & Pushdown Automata	CFG, derivation trees, ambiguity, CFG simplification, Chomsky Normal Form (CNF), Pushdown Automata (PDA).	Leftmost derivation generator, SVG parse tree visualizer, dual-tree ambiguity proof, ε/unit/useless simplification pipeline, and animated vertical stack PDA.

2. Algorithmic Architecture & Mathematical Foundations

2.1 Regular Expressions to Finite Automata (Thompson's Construction)

Thompson's construction converts a postfix regular expression into an equivalent ε-NFA by inductively assembling state sub-gadgets. For concatenation, the accept state of the first operand connects to the start state of the second via an ε-transition. For union, new start and accept states branch and merge around the operands. For Kleene star, backward and forward bypass ε-transitions are established.

2.2 Subset Construction (NFA to DFA)

To convert an ε-NFA $M_N = (Q_N, \Sigma, \delta_N, q_0, F_N)$ to a deterministic automaton M_D, we compute the ε-closure of state subsets. Starting with $S_0 = \epsilon\text{-closure}(q_0)$, each DFA state represents a distinct subset of NFA states. For every alphabet symbol $a \in \Sigma$, the target state is defined by $\delta_D(S, a) = \epsilon\text{-closure}(\text{move}(S, a))$.

2.3 DFA State Minimization (Table-Filling / Myhill-Nerode)

Unreachable states are first pruned via BFS traversal from q_0. The triangular distinguishability table $\mathcal{M}(p, q)$ initializes pairs where exactly one state is accepting. Inductive fixed-point iterations inspect all symbols $a \in \Sigma$: if $\delta(p, a)$ and $\delta(q, a)$ are already marked as distinguishable, then (p, q) is marked.

Unmarked pairs upon convergence form equivalence classes which are merged to construct the quotient DFA.

2.4 Pushdown Automata (PDA) Dynamics

A Pushdown Automaton $M = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$ extends finite-state control with an unbounded LIFO stack. For language $L = \{a^n b^n \mid n \ge 1\}$, the simulator pushes A onto the stack for every a read, and pops A for every b. Acceptance occurs when the input is fully consumed and the stack returns to initial symbol Z_0 in an accepting state.

3. Verification Matrix & Test Record

The project includes 70 rigorous automated tests covering all modules, edge conditions, boundary values, and algebraic identities. All tests pass with 100% success rate:

Verification Area	Number of Tests	Test Description	Status
Module 1 Language Operations	8 Tests	Prefixes, suffixes, reversals, Σ^k generation, union, concatenation, Kleene star, regex formatting.	**8 / 8 PASS**
Module 2 Finite Automata & Minimization	29 Tests	Thompson's NFA construction, `?` operator, literal vowel `'e'` preservation, parallel branching, subset construction, DFA string tracing, Table-filling minimization, and partial DFA TRAP completion.	**29 / 29 PASS**
Module 3 CFG, CNF & PDA	27 Tests	Grammar parsing, leftmost derivations, parse trees, ambiguity detection, ε/unit/useless removal, strict operator CNF conversion, and PDA stack tracing.	**27 / 27 PASS**
Theoretical Labs (Pumping Lemma)	6 Tests	Adversary game evaluations for regular $a^n b^n$ and context-free $a^n b^n c^n$, boundary safety ($p=0, 1$), and exponent $i=1$ identity check.	**6 / 6 PASS**
Total Verified Suite	**70 Tests**	**Complete End-to-End Mathematical & Algorithmic Coverage**	**70 / 70 PASS (100%)**

4. Conclusion

This microproject demonstrates that formal mathematical principles of computer science can be brought to life through interactive, zero-dependency engineering. The resulting visualizer provides students and instructors with an engaging, verified laboratory for exploring automata, grammars, and computation.

5. Academic References

1. Hopcroft, J. E., Motwani, R., & Ullman, J. D. (2006). *Introduction to Automata Theory, Languages, and Computation* (3rd ed.). Pearson / Addison-Wesley.
2. Sipser, M. (2013). *Introduction to the Theory of Computation* (3rd ed.). Cengage Learning.
3. Linz, P. (2016). *An Introduction to Formal Languages and Automata* (6th ed.). Jones & Bartlett Learning.